

ANT-EV Project Setup and Operation Guide

Prerequisites, execution, training boundaries, and operating controls

Purpose. This guide is the single operating reference for the ANT-EV demo. The web application demonstrates station-level charging allocation with local prototype model estimators. Training artifacts may replace those estimators after validation.

1. Prerequisites

Item	Required for	Suggested version
Node.js	Installing and running the Vite/React interface	Current LTS
npm	Package installation and scripts	Bundled with Node.js
Python	Optional model training and PDF utilities	3.10 or later
Git	Optional source management	Current stable

2. Run the web demo

From the project root: **npm install**, then **npm run dev**. Open the local address printed by Vite (normally <http://localhost:5173>). Use the scenario selector to load the 10-vehicle station or emergency commute demo. Add a vehicle with SOC, target, deadline, intake cap, temperature, and priority flags. Run or step the simulation to observe reallocation.

3. Production build check

Run **npm run build**. This performs TypeScript checking and creates the deployable static build in the dist directory. Do not treat a successful front-end build as validation of charger control or battery safety.

4. Model and dataset workflow

Prepared datasets and trainer scripts are under the training directory. M1 and M2 have prepared CSV inputs; the remaining model concepts require validated, compatible datasets and an integration layer before they can be described as trained production models. The UI currently identifies its values as local prototype estimators.

Model	Training purpose	Runtime handoff
M1	Energy/session duration and readiness prediction	energy_need_kwh, duration, confidence
M2	Station demand forecast	forecast pressure / congestion
M3	Battery health/RUL - only with suitable aging telemetry	health risk / confidence
M4	Thermal and charging-stress envelope	safe cap / thermal risk
M5	SOC/temperature trajectories	future-temperature risk
M6	Policy ranking in a validated simulator	strategy value after safety masking

5. Training safety

Use train/test splits by time or vehicle where applicable; keep datasets versioned; persist feature schema, scaler, model version, and metrics; evaluate calibration and failure modes; and do not command real charging equipment from an unvalidated model. Runtime safety constraints must remain independent of learned predictions.

6. Troubleshooting

Port in use: Vite will choose another port; use the printed URL. **Install error:** verify Node LTS, then delete only generated dependency folders if necessary and reinstall. **Blank page:** run `npm run build` and fix the reported TypeScript error. **Power is not 100% used:** all active EVs may be at their safe caps; that is a safety result, not unused allocatable capacity.

7. Project structure

Path	Role
src/components/FocusedWorkspace.tsx	Focused user interface: split, models, emergency, architecture, patent brief, and help.
src/services/engine/stationAllocator.ts	PRISM-ANT safety-mask, priority, capped water-filling, and explanation signals.
src/services/engine/prismAntEngine.ts	Candidate-action evaluation and hard constraint checks.
src/services/models/deepLearningLayer.ts	Prototype inference contract for M1-M6.
training/	Datasets and model-training utilities.

8. Operating boundary

This project is a software demonstration and documentation aid. Integrating with real chargers needs OEM BMS limits, charger protocol conformance, electrical protection, cybersecurity controls, monitoring, audit logging, fail-safe defaults, and specialist safety engineering review.

9. Documentation set

The documentation folder intentionally contains only three PDFs: this project guide, the PRISM-ANT algorithm specification, and the patent specification draft. Older generated documentation is consolidated out of the project root.